

SIH26055 — RL Team Working Brief

Version 1 · 2026-09-05 · Owner: Aamir (itsaamirhashmi@gmail.com)

Read `RL_LANE_HANDOFF.md` too. That document is the *technical and domain* brief: what the problem is, what the metrics mean, why the bar is where it is. **This document is the *operating contract*:** how to set up, how to work in the repository, what to do, and — the part that matters most — **exactly what to hand back.**

0. The shape of this engagement, in one paragraph

Aamir is the one executing in the repository, with Claude Code. You are not expected to land production code on `main`; you are expected to **decide things and produce one trained artefact**. That inverts the usual handoff: your primary deliverable is not a pull request, it is a set of **written decisions with numbers attached**, plus a checkpoint that loads. Anything you leave as "pick whatever seems reasonable" becomes a decision Aamir has to make — and this project's whole discipline (`CLAUDE.md`) is that undocumented decisions are how the previous attempt died. **If it is a choice, write it down. If it is a number, say how you measured it.**

1. What "done" means

A folder named `HANDOVER/` containing the items in §8. That is the deliverable.

The acceptance test is one command. When Aamir can run this and see a row `7 rl` in the table, the RL lane is finished:

```
python -m rfenv.compare --seeds 3 --sampled 10 --figures --out runs/baselines
```

If §8 is complete, he can get there without asking you a single question. **That is the bar for your handover, and it is the only bar.**

PART I — GETTING SET UP

2. The repository

```
git clone https://github.com/OkayWheresAamir/SIH26055.git
cd SIH26055
```

Ask Aamir for collaborator access if the clone 404s — the repo is private.

Read these four files before writing any code. They are short and they are authoritative:

File	Why
<code>CLAUDE.md</code>	The repository's rules. Provenance, authority, what may not be touched.

File	Why
docs/project/SIH26055_PROBLEM_STATEMENT.md	What DRDO actually asked for. Settles scope disputes.
docs/project/EVALUATION.md §4, §5, §7	Metrics, the ladder, the protocol. Do not redefine a metric anywhere else.
docs/project/RL_LANE_HANDOFF.md	Your technical brief.

Then read the module docstring at the top of `rfenv/env.py`. It was written for you and it is the best 15 minutes you will spend.

3. Python environment

Built and tested against **Python 3.12.6** on **macOS arm64**. Linux is fine; 3.11+ should work.

```
python3 -m venv .venv
.venv/bin/pip install --upgrade pip
.venv/bin/pip install -r requirements.txt
```

Verified installed versions in the reference environment (checked 2026-09-05):

Package	Version	Needed for
numpy	2.5.2	everything
h5py	3.16.0	reading the Turing HDF5 files
gymnasium	1.3.0	the L3 agent interface
matplotlib	3.11.1	figures only (<code>rfenv/render.py</code>)
pytest	9.1.1	tests
pymupdf	1.28.2	rebuilding the docsearch index only
markdown, xhtml2pdf	3.10.3, 0.2.18	rendering these handoffs to PDF

Activate it or prefix every command with `.venv/bin/`. Every command in this document assumes one or the other.

3.1 Task 0 — the compatibility check that could eat a day

`stable-baselines3` is **not installed** and is not in `requirements.txt`. Before you plan anything, find out whether your chosen SB3 version supports **gymnasium 1.3.0**, which is what this repository runs. SB3 has historically pinned an upper bound on gymnasium.

```
.venv/bin/pip install "stable-baselines3>=2.3" torch
.venv/bin/python -c "import stable_baselines3, gymnasium; print(stable_baselines3.__version__, gymnasium.__version__)"
```

Three outcomes, and **you decide which one** (this is deliverable **D1**):

1. **It works.** Pin both versions and move on.
2. **SB3 wants an older gymnasium.** Downgrading gymnasium is *allowed* — `requirements.txt` states plainly that nothing is version-sensitive and no result depends on a library version. The freeze list is `rfenv/constants.py`, not `requirements.txt`. **Re-run `pytest tests -q` after downgrading and confirm 229 still pass.** If any fail, stop and tell Aamir.
3. **It is a mess.** Write a single-file PPO (CleanRL-style, ~300 lines, torch only). This is a legitimate answer and for a 36-action discrete env it is not hard. Say so in **D1** and give the reason.

Whichever you choose, **record it with the evidence** — the version numbers you actually ran.

4. The dataset — the part most likely to go wrong

4.1 What it is

The **Turing Synthetic Radar Dataset (TSRD)**, Alan Turing Institute, Apache-2.0. It holds radar **Pulse Descriptor Words** — one row per detected pulse, five columns: **ToA (μ s)**, **Centre Frequency (MHz)**, **Pulse Width (μ s)**, **AoA (deg)**, **Amplitude (dB)**. Confirmed against `metadata/feature_names` in the files themselves.

`data/` is **gitignored**. It will not arrive with the clone.

4.2 Do not re-download it. Ask Aamir for the train half.

This is the single most important paragraph in this section.

The full TSRD train split is **2,500 pulse trains per receiver mode**. This project uses **47 specific scan/stare pairs**, chosen by a stratified rule recorded in `docs/project/RESEARCH_MAP.md` (40 strata by stare file size + 6 extremes + 1 mid-range extra). **A fresh download gives you the wrong 47** unless you replicate that rule exactly, and then every number you produce is incomparable with the ladder in §5 of `EVALUATION.md`.

So: **Aamir ships you the train half**. About **1.1 GB**, over a drive share or an external disk.

He must ship only the train half. Not the test half. This is not a formality — see §7.

```
data/
├── turing/
│   ├── scan/
│   │   ├── train_scan/ 47 files, ~88 MB
│   │   ├── config_2.h5
│   │   ├── config_39.h5
│   │   └── ...
│   └── stare/
│       ├── train_stare/ 47 files, ~997 MB
│       ├── config_2.h5
│       └── ...
```

Optional but worth asking for: `data/cache/contrib/` — 94 `.npz` files, ~8 MB. This is the derived per-emitter contribution cache. Without it, the first pass over the dataset does a cold build at 1–3 s per config (a few minutes, once). With it, episode setup is free. It is safe to copy; it is regenerated from the HDF5 files and nothing depends on it being fresh.

The Hugging Face repository id is not recorded anywhere in this repository — I checked `data/turing/README.md` and `RESEARCH_MAP.md`. Treat the download route as unverified. The dataset is gated on Hugging Face; if you genuinely need to fetch it yourself, get the repo id and the access from Aamir, and be aware you are then responsible for reproducing the 47-file selection.

4.3 The 47 train config ids

```
2 39 59 64 81 92 189 210 252 258 269 279 384 391 398 418 535 594 610 658 706 719
921 940 954 1073 1089 1144 1181 1366 1417 1431 1454 1521 1534 1631 1635 1710 1776
1864 1902 1950 1967 2027 2219 2356 2445
```

Both `scan/train_scan/` and `stare/train_stare/` must contain exactly these 47.

4.4 Verify your copy before you trust it

Save as `check_data.py` **outside the repo** (throwaway checks do not go in this repository) and run it:

```
import h5py
from pathlib import Path

IDS = [2, 39, 59, 64, 81, 92, 189, 210, 252, 258, 269, 279, 384, 391, 398, 418, 535, 594, 610, 658,
       706, 719, 921, 940, 954, 1073, 1089, 1144, 1181, 1366, 1417, 1431, 1454, 1521, 1534, 1631,
       1635, 1710, 1776, 1864, 1902, 1950, 1967, 2027, 2219, 2356, 2445]

for source in ("scan", "stare"):
    d = Path(f"data/turing/{source}/train_{source}")
    got = sorted(int(p.stem.split("_")[1]) for p in d.glob("config_*.h5"))
    assert got == sorted(IDS), f"{source}: expected 47 ids, got {len(got)}"
    with h5py.File(d / "config_2.h5", "r") as fh:
        assert fh["metadata"].attrs["collection_time_s"] == 30.0
        print(source, "OK —", len(got), "files,",
              "features:", [b.decode() for b in fh["metadata/feature_names"][:]])

assert not Path("data/turing/scan/test_scan").exists(), "you have the held-out split — delete it"
print("data OK, and the held-out split is absent. Good.")
```

A note on HDF5, since it may be new to you. An `.h5` file is a little filesystem. It has **groups** (directories — `metadata/`, `metadata/transmitters/`), **datasets** (arrays — `data` is the $N \times 5$ PDW matrix, `labels` says which emitter each pulse came from), and **attributes** (scalar metadata hung off a group — `collection_time_s`). `h5py` gives you all three with dict-like access and slicing that returns NumPy arrays. Nothing loads until you slice it.

5. Confirm the install

```
.venv/bin/python -m pytest tests -q
```

Expect 229 passed. Verified 2026-09-05, ~20 s. If you see fewer, or failures, **stop and tell Aamir** — do not work around it. A red suite here means your copy of the environment is not the one the ladder was measured on, and nothing you measure afterwards is comparable.

If `tests/test_freeze.py` fails specifically, you have changed `rfenv/constants.py`. Revert it.

6. Run the thing once, and look at it

6.1 A 30-second smoke test

```
.venv/bin/python -m rfenv.compare --seeds 1 --configs 2 --rungs recency,round_robin --out runs/scratch
```

Verified output shape (run 2026-09-05):

```
2 rungs x 2 scenarios x 1 seeds = 4 episodes -> runs/scratch
```

```
[ 1/2] replay:train/stare/config_1073      0.0 s
[ 2/2] replay:train/stare/config_1089      0.1 s
```

rung	scheduler	ratio	cTTI (s)	coverage	rate/s
5	recency	0.205 [0.150 0.260]	1.81 [1.01 2.62]	0.950 [0.925 0.975]	0.167
2	round_robin	0.053 [0.050 0.056]	1.09 [0.72 1.46]	1.000 [1.000 1.000]	0.183

paired against round_robin, per episode (same scenario, same seed, same grid)

scheduler	ratio	cTTI	both
recency	100.0%	50.0%	50.0%

6.2 The full ladder, as it was measured

```
.venv/bin/python -m rfenv.compare --seeds 3 --sampled 10 --figures --out runs/baselines
```

1,539 episodes. Takes a while. `runs/` is gitignored — these artefacts are rebuilt, never committed.

6.3 What lands on disk

```
runs/baselines/
■■■ comparison.md      the full table with interquartile ranges – read this
■■■ summary.json       the same numbers, machine-readable
■■■ metrics.json       the three metric families
■■■ pareto.png         ratio vs intercept time, one marker per rung
■■■ timeline_config_2.png band-vs-time, one row per rung, with hits and first intercepts
■■■ discovery_config_2.png distinct emitters found vs time, one line per rung
■■■ recency/
    ■■■ replay_train_stare_config_1073_seed0/
        ■■■ run.json      the episode's scalars ( $\gamma$ ,  $\sigma$ , seed, total_pulses, total_reward)
        ■■■ episode_log.csv one row per slot: band, Y, Z, pulses, levels
        ■■■ emitter_table.csv one row per detectable emitter: first/last intercept, count, bands
```

Everything in `EVALUATION.md §4` is computable from `run.json` + the two CSVs. Nothing else is needed, and nothing else is authoritative.

6.4 Looking at the pictures

```
open runs/baselines/pareto.png      # macOS
xdg-open runs/baselines/pareto.png  # Linux
```

Three figures and what to look for in each:

- **pareto.png** — better is **up and to the left**. Rung 5 is the point you must beat. The two hollow markers are reference lines that read the truth grid; **they are not competitors** and must never be plotted or quoted as if they were.
- **timeline_<config>.png** — one row per scheduler, band on the vertical, time on the horizontal, over the scenario's true occupancy. A tick per dwell start, declared hits marked, and a per-emitter strip on the right that turns solid at first intercept. **This is the picture that will tell you whether your agent has learned anything or is just camping.** Look at it every time you train.
- **discovery_<config>.png** — distinct emitters found against time, with `|E|` as the ceiling. Coverage is one number; this is the path it took there.

Fixed scenarios — `config_81` (1 detectable emitter), `config_2` (19), `config_921` (72) — so two runs' figures lay side by side.

You can also render one scenario's raw truth grid:

```
.venv/bin/python -m rfenv.render config_2 stare /tmp/waterfall.png
```

6.5 Other commands worth knowing

```
.venv/bin/python -m rfenv.baselines      # the ladder, and what the two open-loop rungs spend
.venv/bin/python -m rfenv.validate       # the four environment validation gates (slow)
.venv/bin/python -m docsearch "alert confirm dwell time" -k 8      # search the PDFs
.venv/bin/python -m docsearch "how was the dataset generated" --primary
.venv/bin/python -m docsearch --blind-spots                        # the 8 image-only PDFs
```

docsearch returns a `file.pdf:p.7` citation with every hit. **366 pages of PDF are invisible to grep** — this is the only way to reach them. It answers `no strong match` rather than guessing. Always open the cited page before relying on a number: search locates a page, it does not read mathematics off it.

PART II — HOW TO WORK IN THIS REPOSITORY

7. The five things you must not do

Each of these has already cost this project a withdrawn number, or would invalidate the whole evaluation. They are not style preferences.

1. Never edit `rfenv/constants.py`. It is the freeze list as a file — band geometry, the slot clock, the dwell schedule, `N`, `σ`, `γ`, `PD_POPULATION` — frozen 2026-09-04 as **D42**. `tests/test_freeze.py` pins every value as a literal plus a SHA-256 digest over the whole list, so the suite goes red the moment you touch it. And by **D25**, if it moves, **the environment is re-validated from gate 1 and every baseline is re-run**. If you believe there is a genuine bug in it, that is a conversation with Aamir, not a commit.

2. Never touch the 45 held-out test pairs (D8). Not for a sanity check, not for "just one number", not to see whether it generalises. They are touched **once**, at the very end, after the system is frozen — and that run is Aamir's, not yours. `rfenv/scenario.py` refuses those paths unless an explicit `allow_heldout=True` is passed, and logs every such call. **Do not pass it**. The cleanest protection is not to have the files at all, which is why §4.2 says ship only the train half.

3. Never train or benchmark on a scan replay (D36). A scan recording holds only the pulses Turing's own sweeping receiver was tuned to, so a grid built from one hands any sweeping scheduler its answer — measured, interception ratio **0.9999** and censored intercept time **0.00 s**. `rfenv/compare.py` raises `ScanReplayRefused` rather than trusting you. Train on **sampled scenarios**; evaluate on **stare replays + sampled scenarios**.

4. Never let a policy read info unguarded (D29). Wrap every policy in `baselines.guarded()`. It strips info down to `OBSERVABLE_INFO = {"slot", "time_s", "band", "dwell_slots", "Y"}` before the policy sees it, so a policy reaching for `Z`, `pulses`, `first_intercept`, `detectable`, `n_detectable`, `total_pulses`, `newly_intercepted` or `emitter_table` gets a **KeyError in a test** instead of a suspiciously good score in a results table.

5. Never report a single metric. Interception ratio, censored intercept time **and** emitter coverage go together, in every table, always (D14, `EVALUATION.md` §4). A camper scores 0.209 ratio and looks strong until you see 9.67 s and 0.497 coverage beside it. **A reward is not a metric** — average reward is comparable only within one reward family and never ranks across families (D7).

8. Git workflow

Aamir owns `main`. He stages and commits; you propose.

```
git checkout -b rl/ppo-baseline      # branch names: rl/<topic>
# ... work ...
git add rfenv/rl.py tests/test_rl.py
git commit -m "Add the PPO training env and the RLScheduler adapter"
git push -u origin rl/ppo-baseline
```

Then open a PR into `main` and tag Aamir. **Do not merge your own PR.**

Rules:

- **One thing per commit.** Do not bundle unrelated changes.
- **Never commit** anything under `data/`, `runs/`, `.venv/`, or `__pycache__/`. All are gitignored; if `git status` shows them, something is wrong with your setup.
- **Never commit a model checkpoint to git.** A `.zip` of weights is binary and will bloat the repository permanently. Checkpoints travel in the `HANDOVER/` folder, over a drive share.
- **Do not edit docs/project/DECISIONS.md directly.** Aamir merges decisions into it, and concurrent edits to a 2,274-line file are a guaranteed conflict. Write yours into `HANDOVER/DECISIONS_DRAFT.md` instead (§18 gives the format).
- **Do not edit EVALUATION.md, ENVIRONMENT_SPEC.md, PROJECT_ARCHITECTURE.md or CLAUDE.md.** Those are Aamir's, and `EVALUATION.md` is the single authority on metrics — a metric redefined in two places is exactly the failure this repository exists to prevent.
- **Files you may freely add:** `rfenv/rl.py`, `tests/test_rl.py`, anything under a new `training/` directory of your own, and `HANDOVER/`.
- **The one file you must edit carefully:** `rfenv/baselines.py`, to add one `Rung(...)` entry to `LADDER`. Nothing else in that file. Keep the diff to those five lines plus an import.

9. Provenance — the rule that governs every number you report

From `CLAUDE.md`, and it applies to you:

1. **Every factual claim traces to a source you can name** — a specific HDF5 field, a specific page of a specific PDF, or a command whose output you can show.
2. **State how you know.** "I ran `compare.py` and read `summary.json`" and "the paper asserts" are different claims. Say which one you are making, every time.
3. **A measurement you did not run this session is not a fact.** Re-run it, or label it **unverified** — use the word, do not hedge.
4. **A summary of a source is not a second source.** Cite the primary.
5. **Nothing is imported from ../SIHProto.** That repository's code, documents and numbers are deliberately absent here and none of it is established fact.

Practically: **when you give Aamir a number, give him the command that produced it in the same breath.** He cannot check a plausible-sounding wrong claim, and he should not have to.

PART III — THE CODE, IN THIRTY MINUTES

10. What the environment is

Four layers, one module each, all built and frozen:

Layer	File	What it does
L0 scenario	<code>rfenv/scenario.py</code>	Reads the HDF5 files into per-emitter contributions. Replays a recording, or samples a fresh mix from the train pool.
L1 truth grid	<code>rfenv/truth.py</code>	The 36×600 (<code>band, slot</code>) → S/C/Z grid. Z = "was this cell occupied", C = pulse count, S = peak level.
L2 receiver	<code>rfenv/receiver.py</code>	The noise draw. Turns a true level S into a binary declaration Y at the frozen threshold γ .
L3 agent	<code>rfenv/env.py</code>	The Gymnasium environment. This is your whole interface.
artefacts	<code>rfenv/metrics.py</code> , <code>rfenv/render.py</code>	Episode logs, the three metric families, the figures.
ladder	<code>rfenv/baselines.py</code> , <code>rfenv/compare.py</code>	The seven schedulers, two reference lines, and the runner.

11. The interface, exactly

```
from rfenv.env import ScanEnv
from rfenv.scenario import EmitterPool

env = ScanEnv(pool=EmitterPool.from_train(), reward="hit_z") # pool, never scenario
obs, info = env.reset(seed=0)
obs, reward, terminated, truncated, info = env.step(action)
```

Action space: Discrete(36). Choose one of 36 frequency bands to listen to. **All 36 are legal at every step — no action masking is needed.**

Observation space: Box(0.0, 1.0, shape=(109,), dtype=float32). That is 36×3 + 1, ratified as **D34**. Every component is natively a fraction, so **no scaling layer is needed anywhere**:

Slice	Index	Meaning
HIT_RATE	0:36	declared hits per slot looked, per band
VISIT_DENSITY	36:72	fraction of airtime spent on that band (sums to 1)

Slice	Index	Meaning
STALENESS	72:108	$(t - \text{last visit}) / 600$; a never-visited band reads 1.0
CLOCK	108	normalised episode time

Named constants for these slices live in `rfenv/baselines.py` — **use them, never a bare integer**. A test asserts they still describe what `env.py` builds.

Nothing truth-side appears in the observation. `Z`, per-emitter levels and `first_e` are all available to the *reward* and all absent from this vector. That asymmetry is D29 and it is the reason the comparison means anything.

Reward: one of three candidates, all **per slot**, and a dwell's reward is the **sum over its slots** (D31), so a 100 ms dwell can earn up to +2:

Key	What it pays for	Predicted to favour
<code>hit_z</code> (<i>default</i>)	+1 per true occupied cell — prices opportunity	interception ratio
<code>hit_y</code>	+1 per declared hit — weights cells by loudness	censored intercept time
<code>first_intercept</code>	+1 per emitter intercepted for the first time	coverage / discovery

The set stays at three (D29). Do not invent a fourth. Every candidate scored on the same scenarios is another draw, and best-of-many is partly selection noise.

12. The three things that will trip you up

1. An episode is 600 slots, not 600 steps. Seven of the 36 bands carry a 100 ms dwell and consume two slots; the other 29 carry 50 ms and consume one. So an episode runs **300 to 600 `step()` calls** while always covering exactly **30 s**. Airtime is the only currency; retuning is free.

What this costs you: anything computed per **step** — advantage normalisation, entropy schedules, "average reward per step", `n_steps` rollout boundaries — is measuring something different from what the metrics measure. Episode lengths vary within a batch. Budget for this.

And the naive reward is wrong: "+1 per dwell regardless of length" would make the seven wide bands strictly dominated, and those are measurably the bands holding the densest emitter populations and the slowest rotators.

2. terminated, not truncated (D35). The 30 s horizon is the task definition — Turing's own `collection_time_s`, and the extent of the world the grid describes — not an artificial cap. There is no state beyond slot 600 to bootstrap a value from. **Do not bootstrap at the end of an episode.**

3. γ_{RL} matters more than usual. A 30 s episode with up to 600 steps is long enough that $\gamma_{\text{RL}} = 0.99$ gives a ~100-step horizon — **shorter than an episode**. An agent that cannot see the end of the episode cannot trade early exploration against late exploitation, which is the whole task. This is one of the three things worth sweeping (§15).

13. Where rung 7 plugs in

Exactly one place. Nothing in `env.py`, `metrics.py` or `compare.py` needs to change.

```
# rfenv/rl.py
class RLScheduler:
    """A trained policy as a `(obs, info) -> band` callable, per metrics.run_episode."""
    def __init__(self, model):
        self.model = model

    def __call__(self, obs, info) -> int:
        action, _ = self.model.predict(obs, deterministic=True)
        return int(action)

# rfenv/baselines.py — one entry appended to LADDER
Rung("rl", "7", "RL scheduler",
    "Ours. Trained on sampled scenarios; sees only the D34 observation.",
    lambda rng, grid: RLScheduler(load_checkpoint()))
```

`baselines.make()` wraps every deployable rung in `guarded()` automatically, so your policy is held to the same information constraint as every other rung.

14. Your target

Measured 2026-09-04, 1,539 episodes, $\gamma = -111$ dB, $P_{fa} = 1.3499e-3$, reward hit_z. From EVALUATION.md §5.

#	scheduler	ratio	cTTI (s)	coverage	beats RR on both
1	random	0.0669	4.16	0.858	42.1%
2	round-robin (equal airtime)	0.0605	4.18	0.865	—
3	Turing reference sweep	0.0805	3.74	0.864	51.5%
4	greedy camper (observation-fed)	0.2088	9.67	0.497	1.8%
5	recency / activity — the bar	0.1104	3.20	0.897	70.2%
6a	Apfeld: Active RFs	0.1320	4.32	0.860	53.2%
6	Apfeld adaptive (no tracking)	0.2455	14.86	0.367	4.1%
—	camper, truth-fed (reference line)	0.5680	15.71	0.296	7.0%

#	scheduler	ratio	cTTI (s)	coverage	beats RR on both
—	pulse-capture oracle (<i>reference line</i>)	0.6579	8.01	0.691	19.3%

The bar is rung 5, not round-robin. `argmax(hit rate + gap in sweeps)` — four lines of code — Pareto-dominates the floor on 70.2% of episodes.

Three levels of success:

- **Minimum.** Beat round-robin on both metrics on >70.2% of paired episodes. Failing this means something is structurally wrong, not that RL is hard.
- **The real result.** Pareto-dominate rung 5: ratio > 0.1104 **and** cTTI < 3.20 s, paired per episode.
- **The claim worth making.** Beat rung 6 on ratio *without* its 14.86 s intercept-time collapse.

The two reference lines are not competitors and never appear unlabelled. `oracle_pulse` is a ceiling for interception ratio **only** — measured, it *loses* censored intercept time to plain round-robin on 80.7% of episodes.

PART IV — THE WORK

15. Day by day

Six days, three roles. **If you are two people, R1 and R3 merge — they share the training harness anyway. R2 stays separate.** The person who trains the agent must not be the only person who evaluates it; that is the single most common way a hackathon ML result turns out to be wrong.

Role	Owns
R1 — Training	the algorithm, the loop, hyperparameters, checkpoints, <code>RLScheduler</code>
R2 — Evaluation	rung 7 in <code>LADDER</code> , the comparison runs, seeds, the figures, the leak test
R3 — Reward & observation	the three reward candidates, D47's rule, the D30 experiment

Prerequisites for everyone: §3, §4, §5 and §6.1 done. You have seen a table print.

Day 1 — get an agent to move, and get it measured

#	Task	Owner
1.1	Resolve Task 0 (§3.1). Pin versions. Confirm <code>pytest tests -q</code> still gives 229.	R1

#	Task	Owner
1.2	<code>rfenv/rl.py</code> — <code>make_train_env()</code> returning <code>ScanEnv(pool=..., reward=...);</code> Gymnasium's <code>check_env</code> clean	R1
1.3	Train PPO for a token number of steps on <code>hit_z</code> . Tune nothing.	R1
1.4	<code>RLScheduler(checkpoint)</code> — a (<code>obs, info</code>) -> <code>int</code> callable, wrapped in <code>guarded()</code>	R2
1.5	Add <code>Rung("rl", "7", ...)</code> to LADDER; <code>compare.py</code> picks it up with no other change	R2
1.6	Read <code>DECISIONS.md</code> D28, D29, D31, D34, D46. Start the D30 plan.	R3

Day 1 ends with a bad agent that is fully measured. That is the point. An untuned PPO scoring below random, appearing correctly in the §14 table, is worth more on Day 1 than a good agent you cannot score.

Day 2 — make it actually learn

#	Task	Owner
2.1	Vectorised training (8–16 envs), longer run, learning curve logged	R1
2.2	Sweep three things and nothing else : <code>γ_{RL}</code> , entropy coefficient, <code>n_steps</code> . ~6 runs.	R1
2.3	Evaluation harness: N seeds × the §5 scenario set, paired against <code>recency</code>	R2
2.4	Train one agent per reward candidate (<code>hit_z</code> , <code>hit_y</code> , <code>first_intercept</code>)	R3
2.5	The leak test: assert the policy's input carries no truth-side component	R2

Gate: does rung 7 beat round-robin on both metrics on >50% of paired episodes? If not, stop tuning and go back to 2.1 — something is structurally wrong, not under-trained.

Day 3 — beat the bar, and find out what is carrying the gain

#	Task	Owner
3.1	Longest training run you can afford; checkpoint often	R1
3.2	Full ladder run with rung 7 in it, <code>--seeds 3 --sampled 10</code> <code>--figures</code>	R2
3.3	Apply D47's rule across the three rewards; record the choice and why	R3
3.4	The D30 experiment — train with and without AoA/PulseWidth; close D30 either way	R3
3.5	Ablation: rung 7 vs rung 5 vs rung 6a — one paragraph on which component earns the gain	R2

Day 4 — freeze

#	Task	Owner
4.1	Freeze: one checkpoint, one reward, one observation spec. Nothing moves after this.	all
4.2	Assemble HANDOVER/ per §16 and §17	all
4.3	Regenerate every figure from the frozen system	R2
4.4	Write the decision drafts (§18)	all

The held-out run is NOT on your list. 45 test pairs, once, after the freeze — **that is Aamir's to run**, and only after **HANDOVER/** is in his hands. Do not run it. Do not ask for the files. If the held-out set is touched before the freeze, the final evaluation means nothing and we cannot honestly claim it.

Day 5–6 — buffer and demo

#	Task	Owner
5.1	A live demo path: load checkpoint → run one episode → render the timeline. Under 30 s.	R1
5.2	Record a backup demo video. The most-cited hackathon failure is a live demo that didn't run.	R1

#	Task	Owner
5.3	Write the negative-results list (E-7)	all

What to skip entirely

Model-based RL, offline RL, multi-agent, transformers, curriculum learning, hyperparameter search frameworks, reward shaping beyond the three candidates, custom network architectures. **Six days.** Default to PPO with library defaults and spend your time on measurement instead.

PART V — THE DELIVERABLES CHECKLIST

16. Everything you must hand back

This section is the contract. Tier 1 items are the ones without which Aamir cannot finish the lane. Tier 2 items are what make the result claimable. Tier 3 is pitch material.

Every decision below must state the value, the evidence, and the alternatives rejected. A decision with no "why" is not reusable, and "why" is what the SIH deck is made of.

16.1 Decisions — thirteen, in writing

ID	Decision	Tier
D-1	Algorithm and library, with pinned versions. Which PPO, which SB3/torch versions, and the outcome of the gymnasium 1.3.0 compatibility check (§3.1). If you downgraded gymnasium, say so and confirm 229 tests still pass.	1
D-2	Policy network architecture. <code>net_arch</code> , activation, whether the value head is shared. Anything other than the library default needs a reason.	1
D-3	The full hyperparameter set, as exact numbers. <code>γ_{RL}</code> , learning rate, entropy coefficient, <code>n_steps</code> , <code>batch_size</code> , <code>n_epochs</code> , <code>clip_range</code> , <code>gae_lambda</code> , <code>vf_coef</code> , <code>max_grad_norm</code> , <code>total_timesteps</code> . A range is not an answer — a range means Aamir picks, which is the thing this document exists to prevent.	1

ID	Decision	Tier
D-4	The training scenario draw. <code>pool=EmitterPool.from_train()</code> confirmed, <code>n_envs</code>, vec-env type (SubprocVecEnv vs DummyVecEnv), the training seed, and how to reproduce the exact run.	1
D-5	Which reward. One of <code>hit_z</code>, <code>hit_y</code>, <code>first_intercept</code>, chosen by D47's rule: the candidate beating round-robin on <i>both</i> headline metrics on the most paired episodes. Within 5 pp, nothing is selected — report both and escalate. Show the <i>both</i> column that decided it.	1
D-6	Inference-time action selection. <code>deterministic=True</code> or sampled? This changes the reported numbers, so it is a decision, not a detail. State it and use the same setting everywhere.	1
D-7	Checkpoint-selection criterion, fixed <i>before any run.</i> Which checkpoint is "the" one, judged on train scenarios only. Picking the best checkpoint after seeing evaluation numbers is the same failure as choosing a gate threshold after the measurement (D39) — and a judge will ask.	1
D-8	D30 — do AoA and PulseWidth enter the observation? Yes or no. With the paired comparison that decided it. If yes, you also owe: the exact fixed-width encoding, the new observation length, which files change, and confirmation that <code>tests/test_baselines.py</code>'s slice assertions still pass. D30 is explicitly owned by the RL lane and is currently the only OPEN decision blocking the freeze.	1
D-9	Observation preprocessing — any wrappers? If you used <code>VecNormalize</code>, frame stacking, or any observation transform, the statistics become part of the policy and must ship with the checkpoint and be applied identically at evaluation. Say explicitly if the answer is "none, the observation is already in [0,1]" — that is the expected answer and it is the simplest one.	1

ID	Decision	Tier
D-10	Evaluation configuration for rung 7's row. How many seeds, which scenario set, which <code>--rungs</code> , so the row is reproducible by one command. Default to <code>--seeds 3 --sampled 10</code> so it matches the existing table.	1
D-11	Stop/fail criteria and the fallback. "If rung 7 has not cleared X by end of Day 3, we ship rung 5 as the headline and report rung 7 as an honest negative result." Agree this in advance , so nobody has to decide it under pressure.	2
D-12	Compute envelope. Where it trained, on what hardware, wall-clock time, so Aamir knows whether he can reproduce it on a laptop or not.	2
D-13	Every deviation from this brief. If you did something differently, that is fine — but it must be written down, with the reason. Silent deviation is the one unrecoverable failure mode.	1

16.2 Code — five items

ID	Item	Tier
C-1	rfenv/rl.py — <code>make_train_env()</code> , <code>RLScheduler</code> , <code>load_checkpoint()</code> . Must import cleanly with no training dependencies beyond what D-1 pins.	1
C-2	The training script and its exact command line , verbatim, copy-pasteable. Not "roughly this" — the actual string you ran.	1
C-3	The baselines.py diff — the one <code>Rung(...)</code> entry plus its import. Five lines. Nothing else in that file.	1
C-4	The requirements.txt delta , with pinned versions and a one-line comment each, matching the style already in that file.	1

ID	Item	Tier
C-5	tests/test_rl.py — at minimum two tests: (a) the checkpoint loads and plays a full episode without a KeyError , and (b) the leak test — a policy reaching for a truth key raises. Both must pass in the 229+ suite.	1

16.3 Artefacts — four items

ID	Item	Tier
A-1	The final checkpoint , plus its SHA-256 , the git commit SHA it was trained at, the exact command , and the date . Without all five, the result is not traceable and by this repository's rules it is not a fact.	1
A-2	The three reward checkpoints (hit_z , hit_y , first_intercept) that produced the D-5 comparison. Aamir needs these to re-verify D47's rule.	2
A-3	The D30 pair — the with-AoA and without-AoA checkpoints, if D-8 came out "yes".	2
A-4	Training logs — CSV or tensorboard, whatever you used. Enough to redraw the learning curve.	2

16.4 Evidence — seven items

ID	Item	Tier
E-1	The §14 table with a row 7 in it , produced by compare.py , plus the summary.json and comparison.md that back it. Not retyped — the actual files.	1
E-2	Paired-win percentages for rung 7 against both round_robin (the floor) and recency (the bar), per episode, from compare.paired_wins() .	1
E-3	The learning curve — reward against timesteps, as a figure. The first thing a judge asks to see.	2

ID	Item	Tier
E-4	The reward Pareto table — three candidates × three metrics × the both column, with the 5 pp margin applied, as D47 requires. Publish the full §4 table with spread and the comparison against rung 5 beside the winner; EVALUATION.md says neither is optional.	2
E-5	The D30 ablation table — paired comparison, with and without AoA/PulseWidth.	2
E-6	One paragraph: which component earns the gain. Rung 7 vs rung 5 vs rung 6a. The ladder is the ablation; this is what it says.	2
E-7	The negative-results list. What you tried that did not work, with the number that showed it. Judges reward this and most decks cannot produce it. This project already has four (D36, D43, D44, D45) — add yours.	3

16.5 The five things Aamir must be able to do afterwards

If any of these fails, the handover is incomplete. **Check them yourselves before you hand it over.**

1. `pip install -r requirements.txt` and get a working training environment.
2. `pytest tests -q` and see **231+ passed** (229 plus your two).
3. `python -m rfenv.compare --seeds 3 --sampled 10 --figures` and see **row 7 in the table**.
4. Re-train from scratch with your command and get **materially the same numbers**.
5. Run the held-out evaluation on the 45 test pairs **without touching any of your code or decisions** — because everything is frozen and written down.

PART VI — HANDING IT OVER

17. The `HANDOVER/` folder

One folder, this shape. Everything in §16 lives in it. Ship it over a drive share, not git.

```
HANDOVER/
■■■ README.md           start here: what is in this folder, and the one command to run
■■■ DECISIONS_DRAFT.md  D-1 .. D-13, in the format of §18
■■■ checkpoints/
```

■ ■■■ final.zip	A-1 – the frozen system
■ ■■■ SHA256SUMS.txt	every checkpoint's hash
■ ■■■ hit_z.zip	A-2
■ ■■■ hit_y.zip	
■ ■■■ first_intercept.zip	
■ ■■■ d30_with_aoa.zip	A-3, if applicable
■■■ code/	
■ ■■■ rl.py	C-1 – drops into rfenv/
■ ■■■ train.py	C-2
■ ■■■ test_rl.py	C-5 – drops into tests/
■ ■■■ baselines.diff	C-3
■ ■■■ requirements.delta	C-4
■■■ results/	
■ ■■■ comparison.md	E-1 – the run with row 7 in it
■ ■■■ summary.json	E-1
■ ■■■ paired_wins.md	E-2
■ ■■■ reward_pareto.md	E-4
■ ■■■ d30_ablation.md	E-5
■■■ figures/	
■ ■■■ learning_curve.png	E-3
■ ■■■ pareto.png	regenerated from the frozen system
■ ■■■ timeline_config_2.png	
■ ■■■ discovery_config_2.png	
■■■ logs/	A-4
■■■ NOTES.md	E-6, E-7 – what earns the gain, and what didn't work

HANDOVER/README.md must open with the one command that reproduces E-1, and the SHA-256 of the checkpoint it uses. If a reader has to hunt, the handover has already failed.

18. The format for a decision

Match the house style in docs/project/DECISIONS.md. Aamir merges these in and assigns the real D4x numbers — you use D-1 ... D-13.

```
## D-3 – PPO hyperparameters

**Status:** SETTLED (2026-09-08)

**The decision.**  $\gamma_{RL} = 0.997$ , learning rate  $3e-4$ , entropy coefficient 0.01,
n_steps = 512, batch_size = 2048, n_epochs = 10, total_timesteps = 4,000,000.

**Evidence.** Six-run sweep, logs in `HANDOVER/logs/sweep/`.  $\gamma_{RL} = 0.99$  gives a
~100-step horizon, shorter than a 300-600-step episode, and the agent camped;
0.997 gives ~330 and it explores. Measured: paired-both against round-robin went
from 38% to 74% on the same eval set, `results/sweep.md`.

**Alternatives rejected.**  $\gamma_{RL} = 1.0$  (no discounting) – training was unstable
across all three seeds we tried. Entropy 0.001 – collapsed to a camper by 1M steps.

**How we know.** Every number above is from `HANDOVER/logs/sweep/summary.csv`,
produced by `python train.py --sweep`, run 2026-09-08 on an M2 Pro.
```

That is the whole format: **decision, evidence, alternatives rejected, how we know**. Four paragraphs. Do this for all thirteen and the handover is most of the way done.

19. If it does not work

This is a real possibility and it is planned for. It is not a failure state.

RL on a restless bandit with a 30 s horizon, cold-started every episode, is genuinely hard, and rung 5 — the bar — is a four-line heuristic that already Pareto-dominates the floor on 70% of episodes. If your agent lands between round-robin and rung 5, **that is a publishable result and it belongs in the deck**, provided you can say *why*.

What we need in that case, and it is worth as much as a win:

- The honest table with row 7 where it actually landed.
- The learning curve showing it did learn something.
- **One paragraph on why.** The most likely answers, in order: the horizon is too short for credit assignment; the observation cannot perceive novelty (**which is exactly D30's argument**, and would close D30 "in" with evidence); the reward is misaligned with the metric (D28 puts the two headline metrics on opposite sides of the observability line, so **no single reward is aligned with both**).
- The negative-results list (E-7).

What is not acceptable is a number nobody can reproduce, or a quiet retreat to reporting only the metric that flattered you. This repository exists because a previous attempt accumulated measured claims that could not be traced back to what produced them.

APPENDICES

A. Command reference

```
# setup
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python -m pytest tests -q # expect 229 passed

# smoke test, ~30 s
.venv/bin/python -m rfenv.compare --seeds 1 --configs 2 \
  --rungs recency,round_robin --out runs/scratch

# the full ladder, with figures
.venv/bin/python -m rfenv.compare --seeds 3 --sampled 10 --figures --out runs/baselines

# just your rung against the bar and the floor
.venv/bin/python -m rfenv.compare --seeds 3 --sampled 10 \
  --rungs rl,recency,round_robin --out runs/rl

# a different reward candidate
.venv/bin/python -m rfenv.compare --reward hit_y --seeds 3 --sampled 10 --out runs/hit_y

# inspect
.venv/bin/python -m rfenv.baselines # the ladder and the two cycle budgets
.venv/bin/python -m rfenv.render config_2 stare /tmp/waterfall.png
open runs/baselines/pareto.png

# search the PDFs (366 pages that grep cannot see)
.venv/bin/python -m docsearch "restless bandit scheduling" -k 8
.venv/bin/python -m docsearch "how was the dataset generated" --primary
.venv/bin/python -m docsearch --blind-spots
```

```
# regenerate this document's PDF after editing the markdown
.venv/bin/python -m scripts.md2pdf docs/project/RL_TEAM_HANDOFF.md
```

compare.py flags: `--out`, `--seeds`, `--configs N` (first N stare replays, for smoke tests), `--sampled N`, `--rungs a, b, c`, `--reward`, `--figures`.

B. Glossary

You are new to RF/EW. This is the whole domain, for our purposes.

Term	Meaning
Band	One of 36 frequency slices of the spectrum. The receiver hears exactly one at a time.
Dwell	One listen on one band, for that band's fixed length — 50 ms (1 slot) or 100 ms (2 slots). The action.
Slot	50 ms. An episode is 600 slots = 30 s.
Sweep	One full pass over all 36 bands. Turing's own sweep is 43 slots (2.15 s).
Emitter	A rotating radar. It illuminates you only when its beam sweeps past — which is why this is hard.
PDW	Pulse Descriptor Word. One row per detected pulse: ToA, frequency, pulse width, AoA, amplitude.
Illumination	A moment an emitter was transmitting at you. The denominator of interception ratio.
Z	Truth: was this (band, slot) cell occupied? Reward may read it; the observation may not.
Y	What the receiver <i>declared</i> — a binary detection after the noise draw. The observation may read this.
γ (gamma)	The detection threshold, -111 dBm. Frozen. Not to be confused with γ_{RL} , the discount factor.
Interception ratio	Of all the moments an emitter was illuminating us, what fraction were we listening for.
Censored intercept time (cTTI)	How long after an emitter became detectable did we first catch it — with a 30 s miss counted as 30 s , not dropped.
Emitter coverage	Fraction of detectable emitters found at least once.

Term	Meaning
Camper	A scheduler that parks on one busy band. Scores well on ratio, terribly on everything else. The pathology to avoid.
Restless multi-armed bandit	The right framing, and say it in the deck. 36 arms, each arm's payoff evolving whether or not you pull it, under partial observability. That one sentence tells you what literature to read.

C. Questions you might have

"**Can we change the reward?**" You choose among the three; you do not add a fourth (D29). If you believe a fourth is necessary, that is a proposal to Aamir with evidence, not a commit.

"**Can we change the observation?**" Yes — that is **D30**, and it is explicitly yours. The observation vector is *not* on the freeze list, the validation gates do not read it, and adding AoA costs a retrain of the policy's input layer, not a re-validation of the environment. But it must be a **decision with a measurement**, not a default.

"**Can we change the episode length / band count / dwell times?**" No. `rfenv/constants.py`, D42, enforced by `tests/test_freeze.py`.

"**The tests fail after I installed torch.**" Tell Aamir before working around it. Most likely the gymnasium version moved (§3.1).

"**Which scenarios do we train on?**" Sampled, from the train pool — `ScanEnv(pool=...)`, which draws a fresh scenario every `reset()` so no episode repeats and there is nothing to memorise (D25, D32). **Never a fixed scenario** — that is for the validation gates.

"**How long should training take?**" Unknown, and we will not guess. Report what you measured (D-12).

"**Do we need action masking?**" No. All 36 bands are legal at every step.

"**Can we see the test set, just to check?**" No. Once. At the end. By Aamir. (D8)

*Provenance: every command in this document was run against this repository on 2026-09-05 and its output checked. Every measured figure quoted traces to `docs/project/EVALUATION.md` and `docs/project/DECISIONS.md`, which carry the evidence. The Hugging Face download route in §4.2 is the one item marked *unverified* — the repository id is not recorded anywhere in this project.*